A Comparison of Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML with a Leakage-Aware Evaluation Pipeline

Thai Nguyen Vu
Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam
thaivn_ph@utc.edu.vn

Bui Van Dung
University of Information Technology VNU-HCM, Ho Chi Minh City, Vietnam
dungbv.17@grad.uit.edu.vn

Tri Nhut Do
University of Information Technology VNU-HCM, Ho Chi Minh City, Vietnam
trinhutdo@uit.edu.vn

Van Du Nguyen
Faculty of Information Technology Nong Lam University, Ho Chi Minh City, Vietnam
nvdu@hcmuaf.edu.vn

***Abstract*—Machine-learning-based network intrusion detection systems (IDS) typically process dozens to hundreds of network-flow features, which is computationally expensive and hard to deploy at the edge. This paper presents a systematic comparative study of four dimensionality-reduction strategies on Apache Spark ML: a baseline using all features, selection by Random Forest Importance, selection by SHAP, and PCA. Each strategy is evaluated across eight classification algorithms together with two ensemble methods on the CICIDS2017 dataset. Unlike most prior work, our emphasis falls on the rigor of the evaluation pipeline: (i) we remove the label-leaking feature `destination_port` and deduplicate flows at the feature level before splitting the data; (ii) we fix the comparison configuration *a priori* to avoid selection bias; (iii) we select models on a validation set held out from the training set, together with an in-distribution stability check and a cross-dataset generalization evaluation (between CICIDS2017 and CSE-CIC-IDS2018); (iv) we assess statistical significance using a paired permutation test over multiple resampled training/test splits, together with Nadeau–Bengio corrected confidence intervals and effect sizes; and (v) we add a per-attack-type multiclass evaluation. The experiments find that feature selection cuts the feature count by about 60% while SHAP Top-30 stays statistically equivalent to the full feature set within $\pm 2.3 \times 10^{-4}$ F1. Cross-dataset F1 collapses and, in one direction, is bimodal across identical runs, so we report its two regimes and compare on AUC-PR. The SHAP Top-30 model is exported to the Jetson Orin Nano Super edge device in a companion systems paper.**

***Index Terms*—Intrusion detection, Apache Spark, Feature selection, SHAP, Data leakage**

I. Introduction

Cyberattacks are growing in scale and sophistication, demanding IDS that are both accurate and scalable on large data [1], [2]. Machine learning has become the dominant approach for anomaly-based IDS, but modern network-flow datasets such as CICIDS2017 [3] provide nearly 80 features per flow, which increases training, inference, and memory cost, and that cost is hardest to absorb on an edge device. Dimensionality reduction addresses that cost, but the available strategies, embedded, unsupervised, or explanation-based, measure "importance" in non-equivalent ways, and they are rarely compared under the same distributed pipeline.

A second issue, discussed less often but no less serious, is whether the evaluation pipeline can be trusted. Many studies report near-perfect binary F1 on CICIDS2017, which is usually a sign of data leakage rather than of model capability. *Data leakage* is information unavailable at deployment time entering training or evaluation, inflating scores so they no longer reflect generalization. Labeling errors and duplicate flows in CICIDS2017 are documented and substantially affect evaluation [4]. There are two common sources of leakage. First, `destination_port` encodes the attacked service, since CICIDS2017 aims its attacks at fixed ports, giving the model a port-to-label shortcut. Second, feature-level duplicate flows are not removed by full-duplicate removal, so a random split places the same sample into both the training and test sets. This paper handles both before comparing methods.

Main contributions. (1) A uniform comparison of

RF Importance, SHAP, and PCA over the same eight Spark ML algorithms and two ensembles; (2) Integration of SHAP-based explanation [5] with a direct contrast against RF Importance [6]; (3) A leakage-aware and statistically tested evaluation pipeline: removing the leaking feature, deterministic feature-level deduplication (label-conflicting groups are removed entirely), model selection on a validation set held out from the training set, and a paired permutation test (exact enumeration) over multiple resamplings with Nadeau–Bengio correction and effect size; (4) A per-attack-type multiclass evaluation, exposing the rare-class weaknesses that binary F1 hides; (5) An edge-aware model-selection recommendation: jointly considering F1, the number of features, and model size for the Jetson Orin Nano Super.

II. Related Work

IDS surveys [1], [2], [7] show that both traditional machine learning and deep learning are widely applied on standard benchmarks such as CICIDS2017 [3]. Many works center on one model family, for instance a deep belief network [8] or deep neural networks benchmarked against classical classifiers [9], without a uniform comparison of dimensionality-reduction strategies on the same distributed pipeline. Apache Spark ML [10], [11] supports large-scale processing, but few studies combine feature selection, model explanation, and statistical testing in a single unified experimental framework. Recent edge IDS studies target resource-constrained platforms, for instance ensemble stacking deployed on an NVIDIA Jetson Nano GPU [12] and a metaheuristic-optimized, SHAP-explained detector evaluated under edge-like resource budgets [13].

Feature selection. Variable-selection theory [14] distinguishes three groups: filter, wrapper, and embedded. Random Forest Importance [6] (embedded) and PCA [15] (unsupervised) are two popular directions for IDS but measure different quantities. SHAP [5] provides consistent local explanations and has been used for IDS [16], but is rarely contrasted directly with RF Importance on the same Spark training set.

Concept drift, anomaly, and evaluation. Concept drift [17] affects IDS stability over time. Anomaly-detection surveys [18], [19] systematize unsupervised detectors that are trained on normal data alone, and such detectors, autoencoders in particular, are used both as a pre-filter ahead of a supervised classifier and to reduce feature dimensionality for IDS on edge devices [20], [21]. However, data leakage ("data snooping") and statistical-significance testing (identified as among the most common pitfalls of ML in security [22]) are rarely handled explicitly on CICIDS2017, and we handle both here. Table I summarizes the comparison.

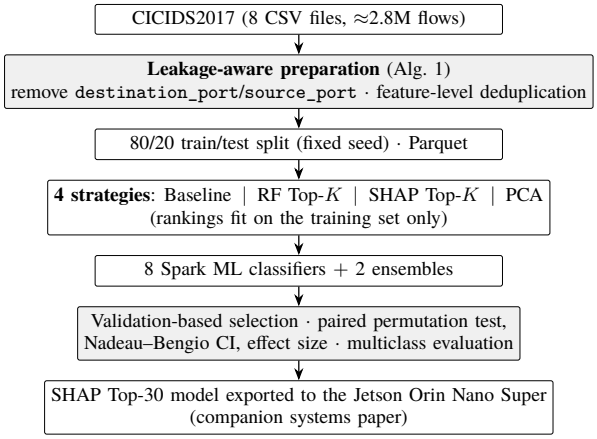


Figure 1. Leakage-aware evaluation pipeline: from raw CICIDS2017 flows to the statistically validated, edge-ready model.

III. Method

A. Overview of the Spark ML pipeline

The offline pipeline consists of sequential steps: data cleaning, deduplication, `VectorAssembler`, `StandardScaler`, an (optional) dimensionality-reduction transform, and then the classifier. Binary labels: Benign (0) and Attack (1). The `fit` of StandardScaler/PCA and the feature ranking use the training set alone, and the test set is the final holdout.

B. Leakage-aware data preparation

We merge the eight CICIDS2017 CSV files, normalize column names, replace infinite/missing values, and then apply two leakage-removal steps *before* splitting the data:

- **Removing the label-leaking feature.** `destination_port` (and `source_port` if present) is removed because it directly encodes the target service of the attack. Keeping it would let the model memorize the port-to-label mapping instead of learning flow behavior. An ablation (keep vs. remove the port) quantifies the impact.
- **Feature-level deduplication (deterministic).** Beyond removing fully duplicate records, we remove flows that are identical across all feature columns before splitting, preventing the same sample from appearing in both the training and test sets. For groups of flows that are feature-identical but have conflicting labels (Benign and Attack, a known error in CICIDS2017 [4]), the entire group is removed rather than arbitrarily keeping one row depending

Table I
CAPABILITY COMPARISON WITH RELATED IDS STUDIES ON CICIDS2017 AND SPARK/EDGE ("—": NOT ADDRESSED; "PARTIAL": DESTINATION_PORT KEPT).

Reference	Dataset	Feature reduction / explainability	Leakage-aware	Statistical testing	Cross-dataset	Multi-class
Khraisat et al. [2]	Multiple (survey)	General	—	—	—	—
Vinayakumar et al. [9]	Multiple (incl. CICIDS2017)	None	—	—	—	Yes
Ferrag et al. [7]	Multiple	None	—	—	—	Yes
Alrawashdeh & Purdy [8]	KDD Cup 99	RBM (unsupervised)	—	—	—	Yes
Sarhan et al. [16]	Multiple (NetFlow, CICFlowMeter)	SHAP	—	—	—	—
Chithra Rani et al. [12]	Multiple (incl. CICIDS2017)	PCA	—	—	—	Yes
Nasir et al. [13]	Multiple	GWO + SHAP	—	—	—	Yes
Hasan et al. [21]	Edge-IIoTset	Autoencoder	—	—	—	Yes
Cantone et al. [23]	CICIDS2017/2018, LycoS	mRMR	Partial	—	Yes	—
The proposed method	**CICIDS2017, CSE-CIC-IDS2018**	**RF/SHAP/PCA**	**Yes**	**Yes**	**Yes**	**Yes**

Algorithm 1 Leakage-aware preparation of CICIDS2017

Input: CSV files $\{F_1, \ldots, F_N\}$; split ratio α=0.8
Output: D_{train}, D_{test} (Parquet)
$D \leftarrow \bigcup_i F_i$ ▷ normalize names; $\pm\infty \rightarrow$ NaN
$D \leftarrow \textsc{DropDuplicates}(D)$; $D \leftarrow \textsc{DropNaN}(D)$
$y_i \leftarrow 0$ if $\textsc{Upper}(\text{label}_i) = \text{BENIGN}$ else 1
$\mathcal{F} \leftarrow$ numeric columns $\setminus$ {`destination_port`, `source_port`, `protocol`, identifiers}
for all groups g of rows identical on all of $\mathcal{F}$ **do**
  if labels within g conflict (Benign and Attack) **then**
    drop g entirely ▷ contradictory ground truth; logged
  else keep one deterministic representative of g
split $D \rightarrow D_{\text{train}}/D_{\text{test}}$ (α, fixed seed); write Parquet

on data partitioning. The numbers of removed groups and rows are reported (270 groups, 44,260 rows), so the deduplication result is deterministic and reproducible.

Algorithm 1 summarizes the procedure, and Figure 1 places it in the full evaluation pipeline. After processing, the data are split into **80% training / 20% test** (with a fixed seed), stored in Parquet format, and reused throughout the experiments. The number of remaining numeric features is 77, and the number of samples after deduplication is 1,785,349/447,275 (2,232,624 total after feature-level dedup).

C. Classifiers and the four dimensionality-reduction strategies

Eight algorithms: Decision Tree, Logistic Regression, SVM (LinearSVC), Naive Bayes, Random Forest, GBT, XGBoost, MLP, together with two ensembles (weighted Hybrid Bagging Top-3 and Soft Voting). LightGBM is excluded because its native library (via SynapseML) supports only x86_64 and cannot run on the Jetson Orin Nano Super edge device (ARM64), the deployment target of this study. XGBoost and GBT represent the gradient-boosting family. The four strategies are **Baseline** (all features), **RF Top-K** and **SHAP Top-K** ($K \in \{20, 30, 40\}$), and **PCA** ($k \in \{20, 30, 40\}$). The RF and SHAP rankings use the training set alone. SHAP is computed on a class-stratified sample so that both benign and attack traffic are represented. PCA is fixed at k=40, more than the Top-30 of the selection methods, because PCA extracts rather than selects: each principal component is a linear combination carrying only part of the variance, so more components are needed to retain an equivalent amount of variance. Of $\{20, 30, 40\}$, k=40 retains the most variance, so it is the fairest point to represent PCA.

D. Hyperparameters and class-imbalance handling

The hyperparameters (Table II) are fixed a priori and shared across all dimensionality-reduction methods, so that the observed differences reflect the feature-selection method rather than per-model tuning. Hyperparameter tuning is separated into a distinct stage (grid search with cross-validation) and applied only to the selected configuration. The stochastic models (RF, GBT, XGBoost, MLP) fix the `seed` for reproducibility. Class imbalance is handled the same way throughout. Every model that supports per-sample weighting, namely Decision Tree, Logistic Regression, SVM, Naive Bayes, Random Forest, and GBT, is assigned class weights according to the benign/attack ratio, equivalent to XGBoost's `scale_pos_weight` mechanism. MLP alone does not support sample weights in Spark ML and is therefore trained without weights, as noted in the limitations.

E. Two-stage comparison design

To avoid selection bias from trying many configurations on the same test set, we separate the *exploration stage* (sweeping $K \in \{20, 30, 40\}$) from the *confirmation stage* (four configurations fixed a priori: Baseline, RF

Table II
MAIN HYPERPARAMETERS OF THE EIGHT CLASSIFIERS (FIXED A PRIORI).

Model	Main hyperparameters
Decision Tree	maxDepth=15, minInstancesPerNode=5, impurity=entropy
Logistic Regression	maxIter=200, regParam=0.001, L2, threshold=0.5
SVM (LinearSVC)	maxIter=200, regParam=0.001
Naive Bayes	gaussian, smoothing=1.0
Random Forest	numTrees=200, maxDepth=15, featureSubset=sqrt
GBT	maxIter=150, maxDepth=6, stepSize=0.05, subsample=0.8
XGBoost	n_estimators=300, max_depth=8, lr=0.05, subsample=0.8, colsample=0.8
MLP	layers=[d,64,32,2], maxIter=150, stepSize=0.01
Class balancing: per-sample class weights (6 models) / positive-class weight scaling (XGBoost), MLP unweighted.	

Top-30, SHAP Top-30, PCA k=40), on which the comparative conclusions rest. No configuration is chosen for its test F1. Within each configuration, the representative model and the tested method pair are chosen by F1 on a validation set (20% held out from the training set, fixed seed), so the test set never participates in selection.

F. Rigorous evaluation pipeline

Metrics. Accuracy, Precision, Recall, F1, AUC-ROC and AUC-PR, plus training/prediction time and model size. Because the data are imbalanced, AUC-PR and attack-class recall are prioritized over accuracy.

In-distribution stability check. Each method's representative model is re-evaluated on a test subset, while distribution shift itself is assessed by the cross-dataset experiment (Section IV).

Statistical-significance testing. The two top methods are trained on N independent resamplings of the training/test split (default N=6), and in each round both use the same split so that the comparison is paired. As a result, the variance reflects the effect of data sampling rather than only of model initialization. The 95% confidence interval is computed using the t distribution ($t_{0.975,N-1}$, not the value 1.96). Because the resamplings overlap in training data, we report alongside it the Nadeau–Bengio [24] corrected confidence interval, with the variance multiplied by $(1/N + n_{\text{test}}/n_{\text{train}})$, and take the corrected interval as the basis for conclusions. Significance is tested by a two-sided paired permutation test on the per-round F1 difference, enumerating all 2^N sign patterns exactly rather than sampling them. Its floor is $p = 2/2^N$, so $N \geq 6$ is needed to reach below 0.05 at all. We report the paired Cohen's d alongside p, and test equivalence directly with a TOST, since a non-significant p is by itself weak evidence for it.

Multiclass evaluation. Because binary classification on CICIDS2017 is almost perfectly separable, we add a per-attack-type multiclass evaluation, comprising per-class precision/recall/F1, macro-F1, and a confusion matrix. It exposes the rare classes (Web Attack, Bot, Infiltration) that binary F1 hides.

IV. EXPERIMENTS AND RESULTS

A. Setup

We use CICIDS2017 with an 80/20 training/test split after leakage removal (Section III-B), Apache Spark 3.4.1 and Java 17, on a cluster of 2× Jetson Orin Nano Super (8 GB) and a coordinator host. The host acts as Spark *master* only and runs no executor, so every training time reported here reflects compute on the two homogeneous Jetson workers rather than the stronger host CPU. Every script logs the run configuration (seed, environment variables, library versions) to ensure reproducibility. The full source code, configuration, and feature lists are publicly available at https://github.com/vuthainguyen1602/Thesis_IDS (branch `develop`). The pipeline can be reproduced on a single machine and does not require an edge-device cluster.

B. Port-feature ablation

Table III quantifies the impact of `destination_port`, using a Random Forest with the Table II hyperparameters, fixed a priori (class-weighted, binary task). After feature-level deduplication, the port's isolated effect is small. Binary F1 is 0.9964 with the port and 0.9954 without it (recall 0.9988 vs. 0.9961, AUC-PR 0.9999 vs. 0.9997), a gap of ≈0.001: the in-domain binary task is near saturation, and feature-level duplicates, the other leakage source, have already been removed. The direction, slightly higher F1 and recall with the port, is nonetheless consistent with leakage. All subsequent results use the port-removed configuration.

That ≈0.001 gap belongs to the tree family, not to the dataset: across all eight classifiers (Table III) the inflation

spans $+0.0007$ (XGBoost) to $+0.0415$ (SVM), because the linear models lean on the port as a service identifier where the tree ensembles recover the same boundary from behavioral statistics. An ablation on one model does not bound the leak for another model class.

Table III
F1 INFLATION FROM DESTINATION_PORT PER CLASSIFIER (BINARY TASK, BOTH ARMS TRAINED ON THE SAME SPLIT).

Model	F1 (no port)	F1 (port)	ΔF1
SVM (LinearSVC)	0.7859	0.8274	+0.0415
Logistic Regression	0.7986	0.8381	+0.0395
Naive Bayes	0.2985	0.3093	+0.0108
MLP	0.9660	0.9711	+0.0052
GBT	0.9924	0.9941	+0.0017
Decision Tree	0.9953	0.9966	+0.0014
Random Forest	0.9954	0.9964	+0.0010
XGBoost	0.9972	0.9979	+0.0007

C. Comparison of the four dimensionality-reduction methods

Table IV summarizes the best model per method, and Figure 2 visualizes F1 for each method $\times$ algorithm pair. The comparison design aims to verify whether feature selection (RF/SHAP Top-30) retains near-baseline performance while cutting ≈60% of the features. PCA k=40 reduces dimensionality in an unsupervised manner at the cost of explainability.

D. Statistical testing (paired resampling)

Table V reports the mean F1 over N=6 resamplings for the two top methods with both 95% intervals, t and Nadeau–Bengio (N–B), and the paired permutation-test p-value. The N–B intervals still overlap. The two methods differ significantly only at the floor of the test's resolution ($p \approx 0.031$, Cohen's $d \approx 1.94$), yet the absolute F1 gap is about 0.0001. A significance test cannot establish equivalence, so we add two one-sided tests (TOST [25]) against a 0.001 F1 margin fixed in advance, on the Nadeau–Bengio error: $p_{\text{TOST}} = 3.9 \times 10^{-6}$, and the tightest margin these six splits support at $\alpha = 0.05$ is $\pm 2.3 \times 10^{-4}$ F1 — equivalence in the positive sense, not mere indistinguishability.

E. Robustness and multiclass evaluation

In the in-distribution stability check (a subset of the test set), the ranking of the methods is preserved (XGBoost leads in all four methods). The multiclass evaluation (Figure 4, Table VI) shows a wide gap between weighted-F1 (≈0.998) and macro-F1 (≈0.806): the rare classes carry the loss. Reading the confusion matrix by where the errors land separates two failure modes. XSS is mostly mislabeled but still detected: 81/111 samples are predicted as Web Brute Force, so at the binary (deployment) level 76% of XSS flows are still flagged as attacks. The damaging mode is leakage into Benign: all 6 SQL Injection samples, 54% of Bot (158/290), and 27% of Web Brute Force are predicted benign. Three mechanisms plausibly drive this: extreme imbalance (6–311 samples vs. 380k benign), short HTTP attack flows that resemble browsing once the port-80 shortcut is gone, and Bot C&C beaconing that resembles background traffic.

F. Cross-dataset generalization

To go beyond *in-domain* evaluation, we train on one dataset and test on the other (cross-dataset evaluation, run in both directions) on the leakage-free feature set shared by the two datasets. Because CSE-CIC-IDS2018 [26] is very large (∼16M flows) we use a label-stratified sample (fraction 0.2, fixed seed): ∼3.14M flows, ∼2.39M after feature-level deduplication, against ∼2.23M for CICIDS2017.

The contrast is the sharpest result in the paper: in-domain F1 is high and stable in both directions (0.9953 and 0.9246, CV <0.1%), yet cross-dataset performance collapses, most clearly in the threshold-free AUC-PR, 0.9997 to 0.4849 (the mean of all 18 runs) and 0.9679 to 0.1926. Cross-dataset F1 sits in two bands for 2017→2018 and around 0.0544 in the other direction, at very low recall (0.073 and 0.030) but still-usable precision (0.772 and 0.272): the model misses the other testbed's attacks rather than alarming indiscriminately. For 2017→2018 that F1 is not even single-valued: the 18 runs split into two disjoint regimes with nothing in between. A mean is therefore the wrong summary: 0.1276 falls in the empty gap and its 95% interval holds 1 of the 18 runs. Of five seeds run twice, three stayed in their regime and two switched, so the distributed fit settles it, not the seed. AUC-PR shows no detectable difference between them (0.4444–0.5163 against 0.4532–0.5397, Welch $|t|$=1.3), so the ranking does not change, only where the 0.5 threshold falls against a cluster of near-identical target flows that crosses it as a block. A single-run cross-dataset F1 only reports which regime the run landed in, so we report the distribution and compare on AUC-PR. The causes are concrete: a different CICFlowMeter version (changed feature scales), a different network configuration, and non-overlapping attack tooling (HOIC/LOIC-HTTP exist

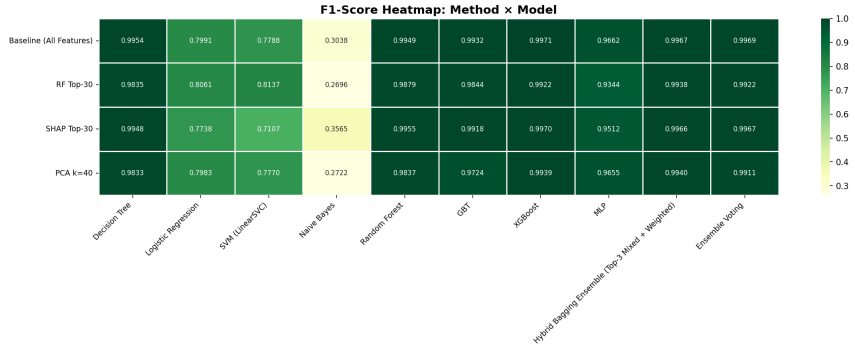


Figure 2. F1 heatmap by method and algorithm (binary, port removed).

Table IV
SUMMARY OF THE BEST MODEL PER METHOD (VALUES FROM THE RELEASED EXPERIMENT OUTPUTS).

Method	Model	F1	AUC-PR	Train (s)	Size (MB)
Baseline	XGBoost	0.9971	0.9999	4615.5	2.32
RF Top-30	XGBoost	0.9922	0.9998	1295.1	2.52
SHAP Top-30	XGBoost	0.9970	0.9999	1226.1	0.003†
PCA k=40	XGBoost	0.9939	0.9998	1444.8	3.82

†Sizes are obtained by serializing each fitted model *on the training cluster*. The distributed model-saving step writes the tree-ensemble data partitions on whichever executor holds them, so entries in the KB range (here 0.003 MB) capture only the driver-local metadata and *under-report* the true size. Since the size of a tree ensemble is governed by the number of trees and nodes rather than by the number of input features, the SHAP Top-30 XGBoost model is in reality comparable to its RF Top-30 counterpart (≈2.5 MB). For the same reason, a reduced feature set can even yield a slightly *larger* model than the baseline (2.52 vs. 2.32 MB): with fewer split candidates the trees need more nodes to reach comparable purity. The single-node export of the deployed pipeline measures model sizes reliably.

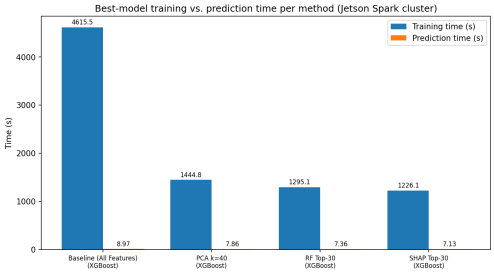


Figure 3. Training and inference time of each method's representative model on the training Spark cluster (Section IV).

Table V
MULTI-RESAMPLING STABILITY AND PAIRED TESTING (VALUES FROM THE RELEASED EXPERIMENT OUTPUTS).

XGBoost	F1 ± t-CI95	N–B CI95	p
Baseline	0.99734 ± 0.00007	± 0.00012	0.031
SHAP Top-30	0.99720 ± 0.00009	± 0.00015	

only in 2018). Is it merely a scale mismatch? Two classic unsupervised adaptations (refitting the scaler on the target, and CORAL [27], both without target labels or retraining) recover no F1 (0.000 either direction, in three runs), so the shift sits in the conditional distribution, not the first two moments. CORAL does lift AUC-PR

Table VI
PER-CLASS PRECISION/RECALL/F1 (RANDOM FOREST, ALL FEATURES).

Class	Support	Precision	Recall	F1
Benign	380,119	0.9981	0.9997	0.9989
DoS Hulk	34,446	0.9990	0.9904	0.9947
PortScan	383	1.0000	0.9530	0.9759
Web Attack – Brute Force	311	0.7346	0.7299	0.7323
Bot	290	1.0000	0.4552	0.6256
Web Attack – XSS	111	1.0000	0.0270	0.0526
Infiltration	12	1.0000	0.6667	0.8000
Web Attack – SQL Injection	6	0.0000	0.0000	0.0000
7 remaining classes, all F1 ≥ 0.95 (see released outputs)				
macro-F1 $= 0.806$		weighted-F1 $= 0.998$		

Table VII
CROSS-DATASET GENERALIZATION (2017 = CICIDS2017, 2018 = CSE-CIC-IDS2018): 18 RUNS OF IDENTICAL CODE AND DATA (13 FOREST SEEDS, FIVE RUN TWICE). THE 2017→2018 F1 IS BIMODAL, SO THAT DIRECTION IS SPLIT INTO ITS TWO REGIMES, AND THE OTHER ROWS ARE MEANS.

Train → Test	F1	AUC-PR
2017 → 2017 (in-domain)	0.9953	0.9997
2017 → 2018 (cross, 11 runs)	0.029–0.085	0.4778
2017 → 2018 (cross, 7 runs)	0.185–0.301	0.4960
2018 → 2018 (in-domain)	0.9246	0.9679
2018 → 2017 (cross)	0.0544 ± 0.0019	0.1926

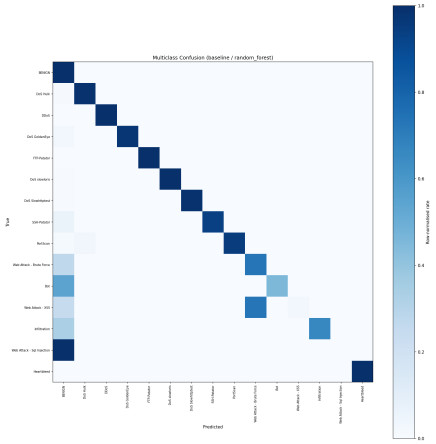


Figure 4. Multiclass confusion matrix (row-normalized).

for 2018→2017 (0.179 to 0.250) with F1 still zero: the ranking improves, the source-fitted 0.5 threshold does not survive, and adaptation here has to recalibrate the threshold as well as the features. Labeling a little of the new network works immediately: retraining on 1% of the target's training split (19k flows) reaches F1 0.92 and 0.99, and pooling them with the 1.79M source flows is worse (0.80, 0.96) than dropping the source data. This collapse is consistent with prior cross-dataset IDS studies [23], [28] and reinforces the central claim of this paper: near-perfect in-domain scores do not certify deployability under distribution shift, so cross-dataset evaluation must be mandatory in IDS validation, and domain adaptation is a prerequisite for practical transfer.

V. Discussion

Method comparison. Baseline and SHAP Top-30 are equivalent within $\pm 2.3 \times 10^{-4}$ F1 (Table V), so secondary criteria decide the deployment choice: explainability, size, stability, feature count. RF Top-30 trails SHAP Top-30 by ≈0.005 F1 at the same feature count, and SHAP Top-30 also yields consistent local explanations for SOC operations.

Bridge to the edge device. Cutting ≈60% of the features reduces vector-assembly cost and model footprint on the Jetson Orin Nano Super, so the selection criterion has to weigh feature count, model size and training/prediction time (Figure 3) alongside F1. Latency and throughput on the Jetson appear in a companion systems paper.

Limitations of the binary evaluation. Removing the port and deduplicating brings binary F1 down to a realistic level, but binary classification remains a relatively easy task. Only under multiclass evaluation do the weaknesses on rare attack classes emerge, and comparing the reduction methods on it is left to future work.

VI. Threats to Validity

Internal validity. The RF/SHAP rankings are derived from the original training set and kept fixed across resamplings, which follows the "a priori configuration" design but lets the ranking see flows that a later resampling puts in its test split. Re-ranking SHAP inside a resampling is expensive, so we did it for two of the six: F1 moved by at most 7.4×10^{-5}, in opposite directions, and 25–26 of the 30 features were reselected. The fixed ranking is not inflating F1. The Top-3 component models of the ensemble are selected on that same held-out validation set, at the cost of training the base models on less data. Class balancing applies to all models except MLP (Spark ML offers no sample weights for it), so the MLP result carries that caveat.

External validity. CICIDS2017 (2017) may not reflect current traffic, and benchmark NIDS datasets carry documented quality limitations [29], so verification on the full CSE-CIC-IDS2018 and newer datasets (CIC-IoT) is needed. The default random split shuffles time, although a time-based split is also supported. The cross-dataset conclusion (Section IV-F) rests on a stratified sample of CSE-CIC-IDS2018, not the whole set. The model assumes an attack distribution approximating CICIDS2017 and has not been tested against deliberately evasive traffic, and feature-level deduplication itself shifts the benign/attack ratio.

Statistical validity. Cross-dataset F1 is not deterministic across runs and is bimodal (Section IV-F), so it is reported over $n{=}18$ runs and each adaptation figure is compared within its own run. The small sample size ($N{=}6$) gives the paired permutation test low power: because the permutation test is two-sided, the smallest attainable p is only $2/2^N$, so $N{=}6$ just barely reaches the 0.05 threshold and can hardly detect small effects. The equivalence claim does not rest on that p-value, however: the TOST of Section III-F tests it directly, against the same Nadeau–Bengio error that accounts for the overlap between resamplings. Increasing N by nested cross-validation is the next step.

VII. Conclusion

This paper compares four dimensionality-reduction strategies for IDS on Spark ML under a leakage-aware and statistically tested evaluation pipeline. The contribution lies as much in how to evaluate reliably as in which method wins: removing the leaking feature,

deterministic feature-level deduplication, model selection on a validation set, exact-enumeration paired testing with Nadeau–Bengio correction, multiclass evaluation, and cross-dataset generalization between CICIDS2017 and CSE-CIC-IDS2018 (Section IV-F). Future work covers an ablation of the deduplication step (the second leakage source), the full CSE-CIC-IDS2018 and newer datasets (RoEduNet, CIC-IoT), transfer of the SHAP Top-30 set between datasets, and real-time edge deployment.

Conflict of Interest

The authors declare no conflict of interest.

References

[1] M. Ring, S. Wunderlich, D. Grüdl, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–167, 2019.

[2] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, pp. 1–22, 2019.

[3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116, 2018.

[4] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and É. Totel, "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes," in *Risks and Security of Internet and Systems (CRiSIS 2022), Revised Selected Papers*, pp. 18–33, Springer, 2023.

[5] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[6] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] M. A. Ferrag, L. Maglaras, S. Moschoyiannis, and H. Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *Journal of Information Security and Applications*, vol. 50, p. 102419, 2020.

[8] K. Alrawashdeh and C. Purdy, "Toward an online anomaly intrusion detection system based on deep learning," in *Proc. 15th IEEE Int. Conf. on Machine Learning and Applications (ICMLA)*, pp. 195–200, 2016.

[9] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525–41550, 2019.

[10] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," in *Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud'10)*, 2010.

[11] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, and A. Talwalkar, "MLlib: Machine learning in Apache Spark," *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.

[12] P. R. Chithra Rani and K. Baalaji, "Deep learning-based ensemble stacking for enhanced intrusion detection in IoT-edge platforms," *Discover Applied Sciences*, vol. 7, p. 928, 2025.

[13] K. Nasir, S. K. Badri, D. M. Alghazzawi, M. Y. Alghamdi, M. Alkhozae, A. Almakky, R. M. Alhazmi, and M. Z. Asghar, "HED-ID: an edge-deployable and explainable intrusion detection system optimized via metaheuristic learning," *Scientific Reports*, vol. 16, p. 2313, 2026.

[14] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003.

[15] I. T. Jolliffe, *Principal Component Analysis*. Springer Series in Statistics, Springer, 2nd ed., 2002.

[16] M. Sarhan, S. Layeghy, and M. Portmann, "Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection," *Big Data Research*, vol. 30, p. 100359, 2022.

[17] J. Gama, I. Zliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.

[18] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[19] L. Ruff, J. R. Kauffmann, R. A. Vandermeulen, G. Montavon, W. Samek, M. Kloft, T. G. Dietterich, and K.-R. Müller, "A unifying review of deep and shallow anomaly detection," *Proceedings of the IEEE*, vol. 109, no. 5, pp. 756–795, 2021.

[20] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, "Conditional variational autoencoder for prediction and feature recovery applied to intrusion detection in IoT," *Sensors*, vol. 17, no. 9, p. 1967, 2017.

[21] T. Hasan, A. Hossain, M. Q. Ansari, and T. H. Syed, "Enhanced intrusion detection in IIoT networks: A lightweight approach with autoencoder-based feature learning," 2025. arXiv:2501.15266.

[22] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *31st USENIX Security Symposium (USENIX Security 22)*, pp. 3971–3988, 2022.

[23] M. Cantone, C. Marrocco, and A. Bria, "Machine learning in network intrusion detection: A cross-dataset generalization study," *IEEE Access*, vol. 12, pp. 144489–144508, 2024.

[24] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003.

[25] D. J. Schuirmann, "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability," *Journal of Pharmacokinetics and Biopharmaceutics*, vol. 15, no. 6, pp. 657–680, 1987.

[26] Canadian Institute for Cybersecurity and Communications Security Establishment, "CSE-CIC-IDS2018 dataset: A collaborative project between the Communications Security Establishment (CSE) and the Canadian Institute for Cybersecurity (CIC)." https://www.unb.ca/cic/datasets/ids-2018.html, 2018.

[27] B. Sun, J. Feng, and K. Saenko, "Return of frustratingly easy domain adaptation," in *Proceedings of the 30th AAAI Conference on Artificial Intelligence (AAAI)*, 2016.

[28] L. D'hooge, T. Wauters, B. Volckaert, and F. De Turck, "Inter-dataset generalization strength of supervised machine learning methods for intrusion detection," *Journal of Information Security and Applications*, vol. 54, p. 102564, 2020.

[29] P. Goldschmidt and D. Chudá, "Network intrusion datasets: A survey, limitations, and recommendations," *Computers & Security*, vol. 156, p. 104510, 2025.